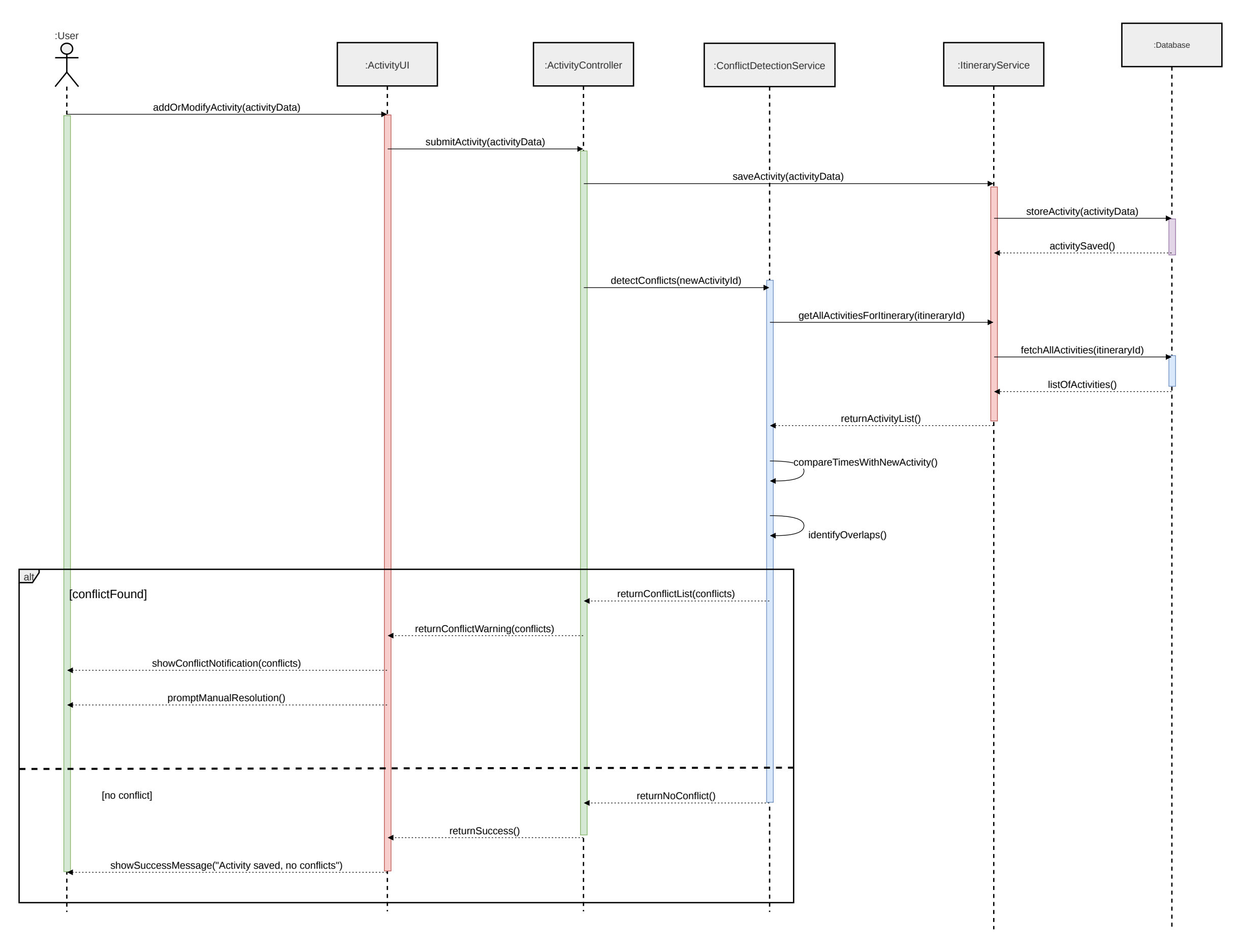
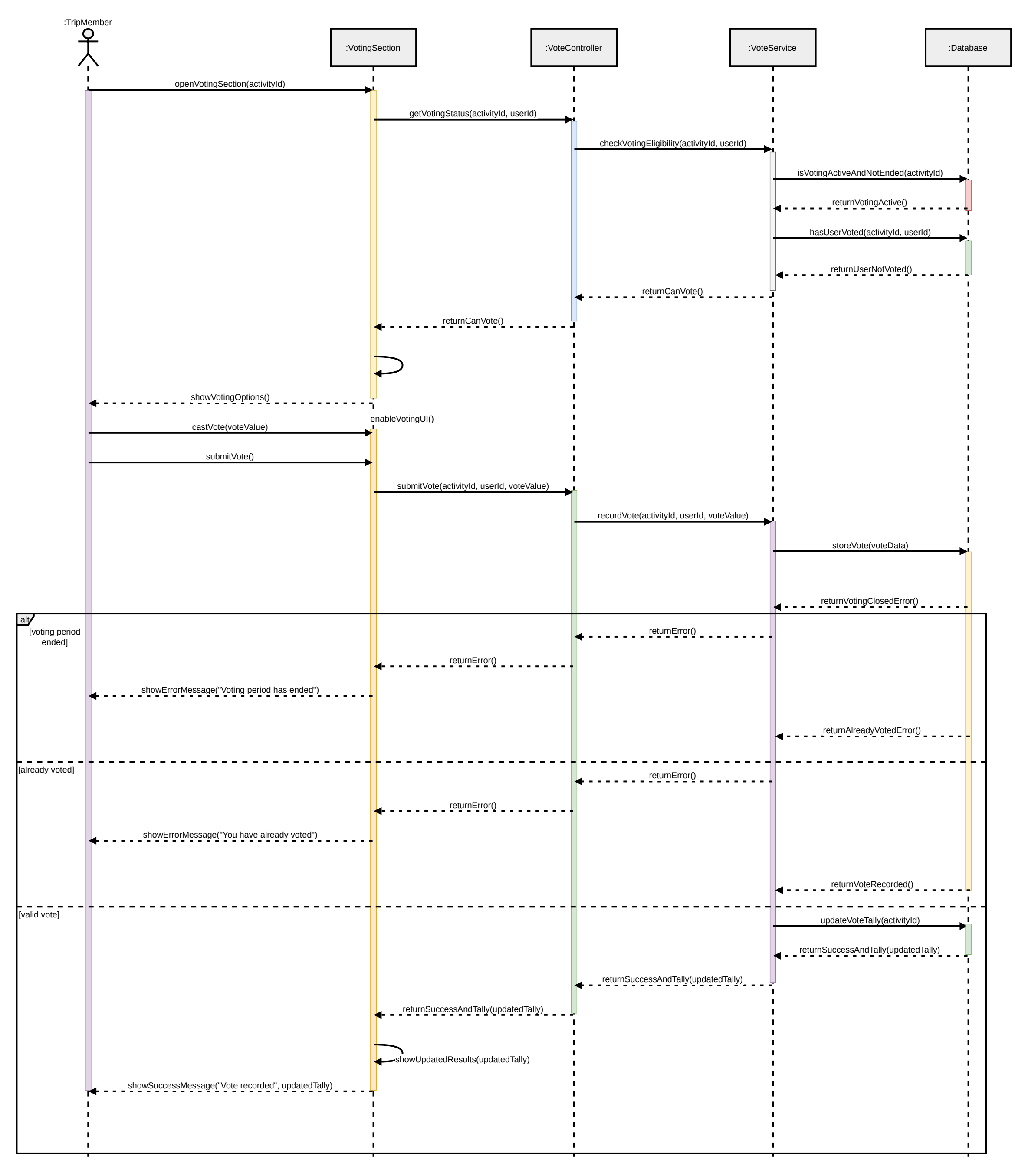
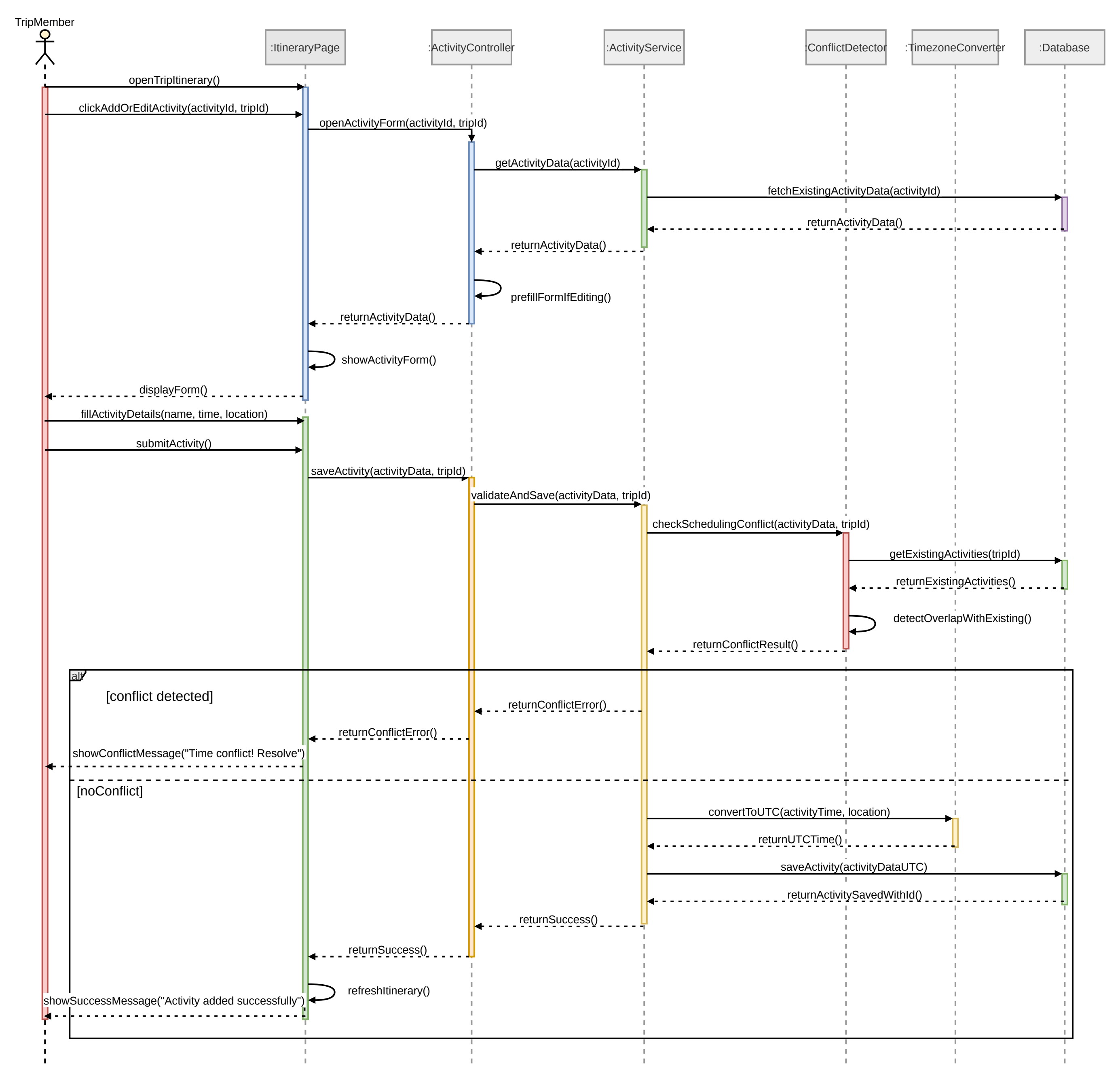
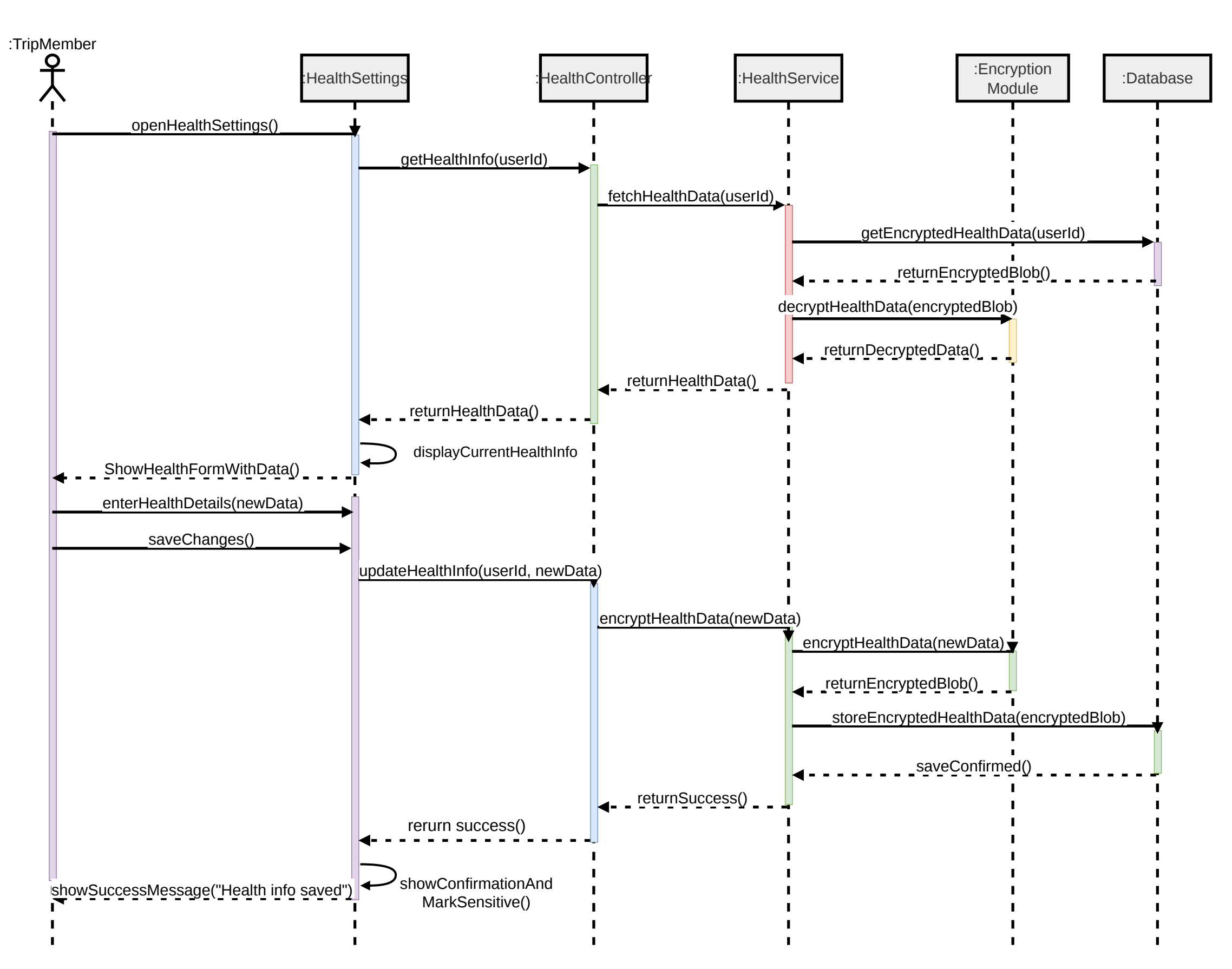
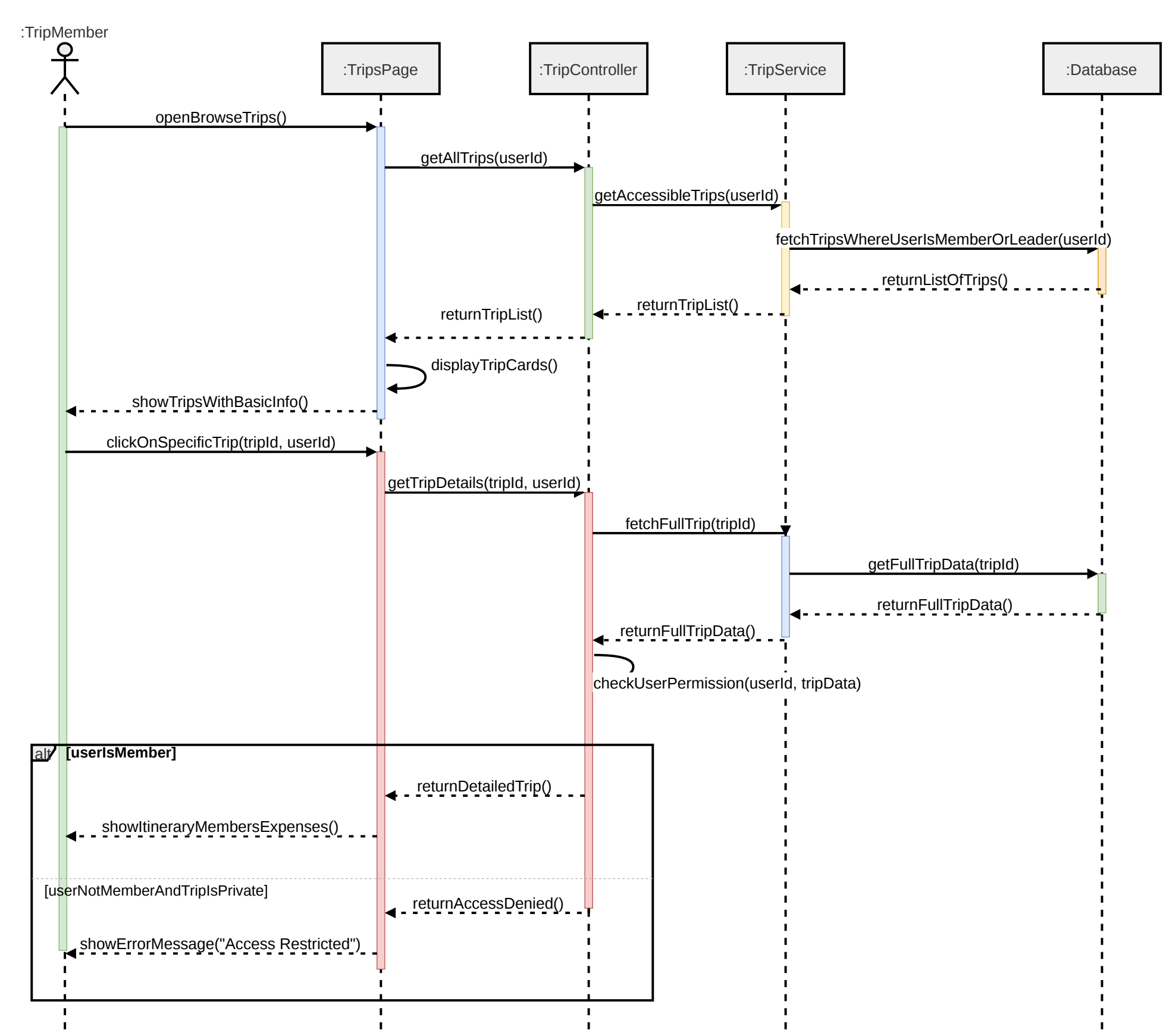
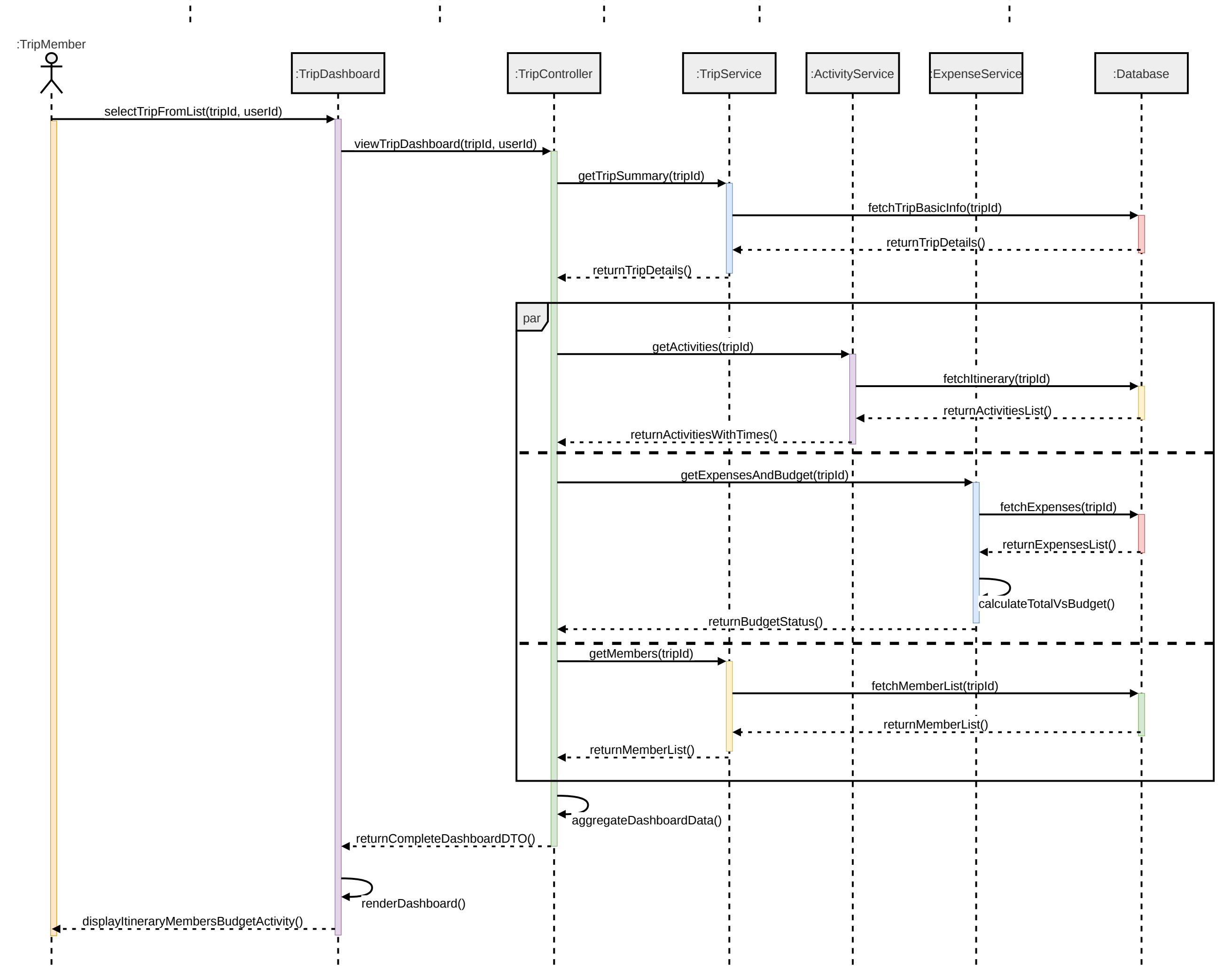

:User
:ActivityUI
:ActivityController
:ConflictDetectionService
:ItineraryService
Database
addOrModifyActivity(activityData)
submitActivity(activityData)
saveActivity(activityData)
storeActivity(activityData)
activitySaved()
detectConflicts(newActivityId)
getAllActivitiesForItinerary(itineraryId)
fetchAllActivities(itineraryId)
listOfActivities()
returnActivityList()
compareTimesWithNewActivity()
identifyOverlaps()
alt
[conflictFound]
returnConflictList(conflicts)
returnConflictWarning(conflicts)
showConflictNotification(conflicts)
promptManualResolution()
[no conflict]
returnNoConflict()
returnSuccess()
showSuccessMessage("Activity saved, no conflicts")

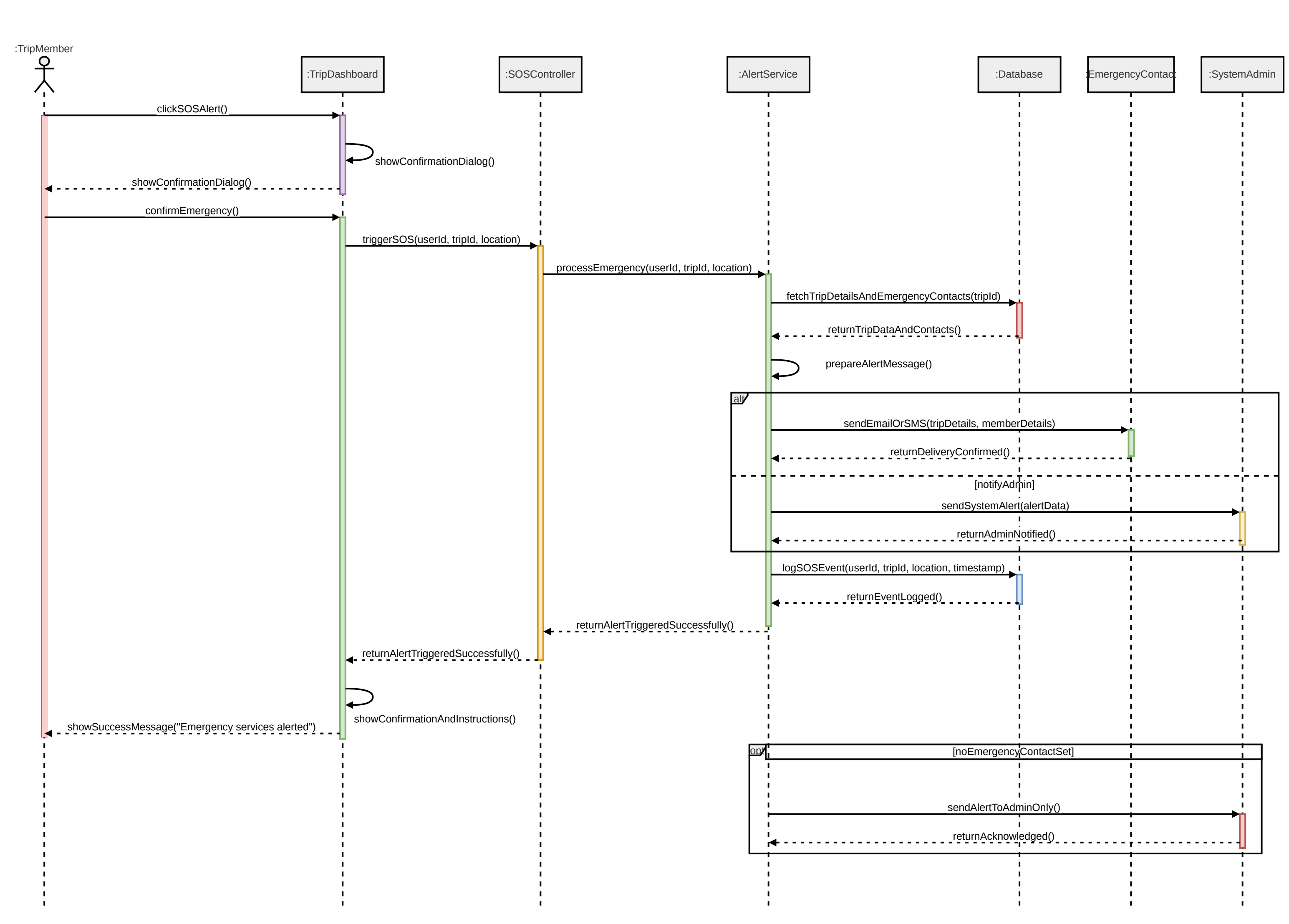

:TripMember
:VotingSection
:VoteController
:VoteService
:Database
openVotingSection(activityId)
getVotingStatus(activityId, userId)
checkVotingEligibility(activityId, userId)
isVotingActiveAndNotEnded(activityId)
returnVotingActive()
hasUserVoted(activityId, userId)
returnUserNotVoted()
returnCanVote()
returnCanVote()
showVotingOptions()
enableVotingUI()
castVote(voteValue)
submitVote()
submitVote(activityId, userId, voteValue)
recordVote(activityId, userId, voteValue)
storeVote(voteData)
returnVotingClosedError()
alt
[voting period ended]
returnError()
returnError()
showErrorMessage("Voting period has ended")
returnAlreadyVotedError()
[already voted]
returnError()
returnError()
showErrorMessage("You have already voted")
[valid vote]
returnVoteRecorded()
updateVoteTally(activityId)
returnSuccessAndTally(updatedTally)
returnSuccessAndTally(updatedTally)
returnSuccessAndTally(updatedTally)
showUpdatedResults(updatedTally)
showSuccessMessage("Vote recorded", updatedTally)

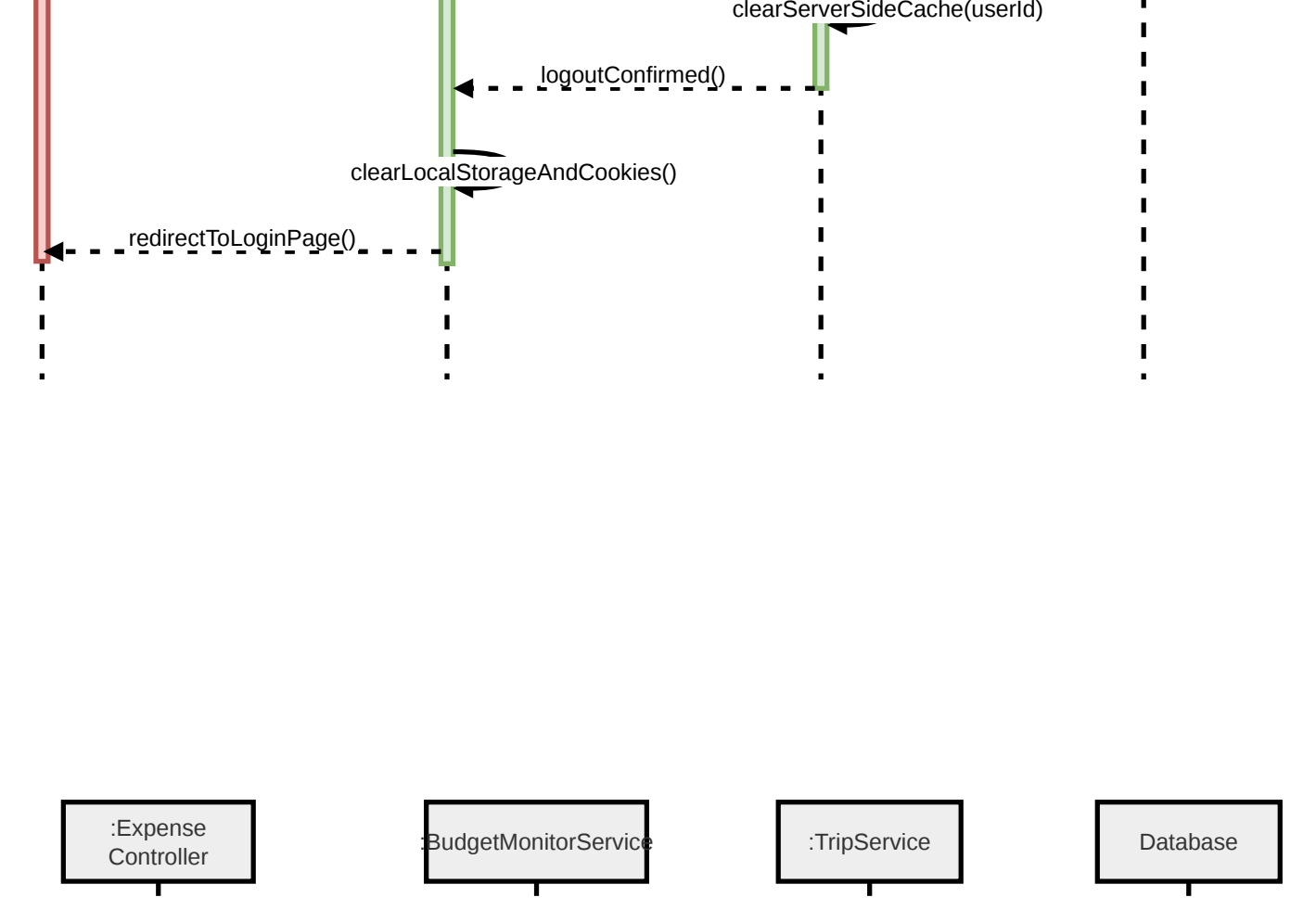

TripMember
:ItineraryPage
:ActivityController
:ActivityService
:ConflictDetector
:TimezoneConverter
:Database
openTripItinerary()
clickAddOrEditActivity(activityId, tripId)
openActivityForm(activityId, tripId)
getActivityData(activityId)
fetchExistingActivityData(activityId)
returnActivityData()
returnActivityData()
prefillFormIfEditing()
returnActivityData()
showActivityForm()
displayForm()
fillActivityDetails(name, time, location)
submitActivity()
saveActivity(activityData, tripId)
validateAndSave(activityData, tripId)
checkSchedulingConflict(activityData, tripId)
getExistingActivities(tripId)
returnExistingActivities()
detectOverlapWithExisting()
returnConflictResult()
alt
[conflict detected]
returnConflictError()
returnConflictError()
showConflictMessage("Time conflict! Resolve")
[noConflict]
convertToUTC(activityTime, location)
returnUTCTime()
saveActivity(activityDataUTC)
returnActivitySavedWithId()
returnSuccess()
returnSuccess()
refreshItinerary()
showSuccessMessage("Activity added successfully")

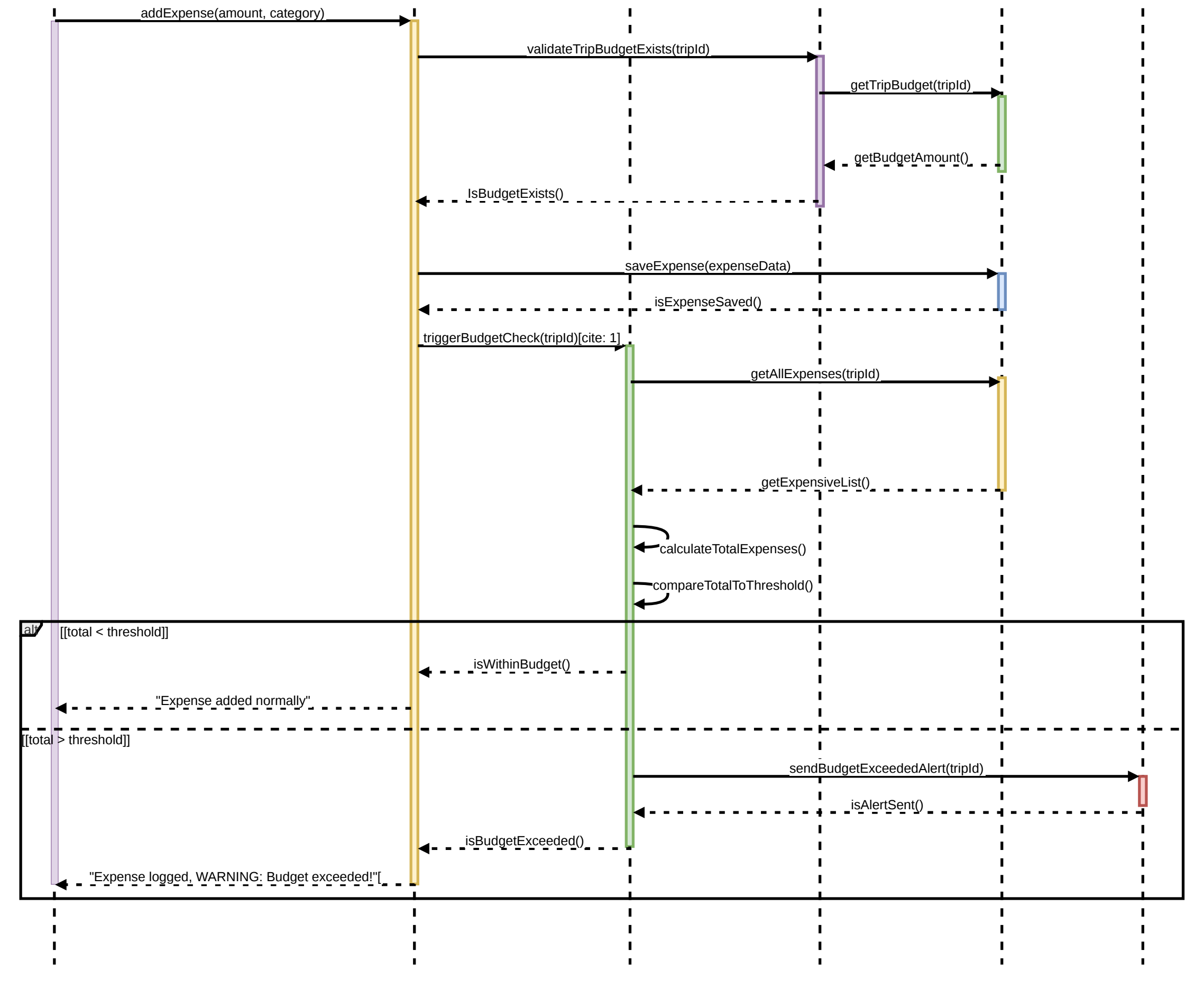

:TripMember
:HealthSettings
:HealthController
:HealthService
:Encryption Module
:Database
openHealthSettings()
getHealthInfo(userId)
fetchHealthData(userId)
getEncryptedHealthData(userId)
returnEncryptedBlob()
decryptHealthData(encryptedBlob)
returnDecryptedData()
returnHealthData()
returnHealthData()
displayCurrentHealthInfo
ShowHealthFormWithData()
enterHealthDetails(newData)
saveChanges()
updateHealthInfo(userId, newData)
encryptHealthData(newData)
encryptHealthData(newData)
returnEncryptedBlob()
storeEncryptedHealthData(encryptedBlob)
saveConfirmed()
returnSuccess()
rerurn success()
showConfirmationAnd MarkSensitive()
showSuccessMessage("Health info saved")

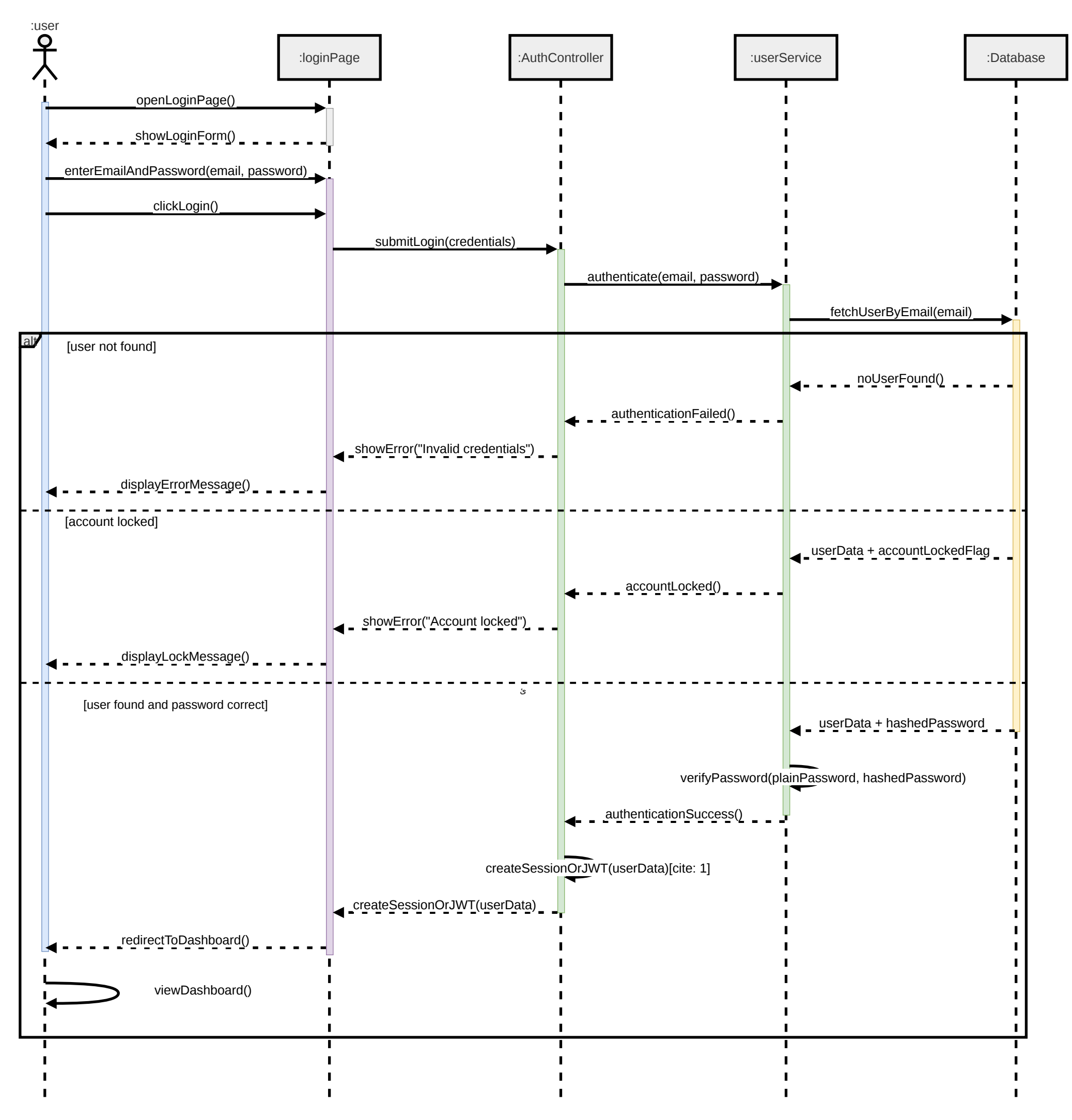

:TripMember
:TripsPage
:TripController
:TripService
:Database
openBrowseTrips()
getAllTrips(userId)
getAccessibleTrips(userId)
fetchTripsWhereUserIsMemberOrLeader(userId)
returnListOfTrips()
returnTripList()
returnTripList()
displayTripCards()
showTripsWithBasicInfo()
clickOnSpecificTrip(tripId, userId)
getTripDetails(tripId, userId)
fetchFullTrip(tripId)
getFullTripData(tripId)
returnFullTripData()
returnFullTripData()
checkUserPermission(userId, tripData)
alt
[userIsMember]
returnDetailedTrip()
showItineraryMembersExpenses()
[userNotMemberAndTripIsPrivate]
returnAccessDenied()
showErrorMessage("Access Restricted")


:TripMember
:TripDashboard
:TripController
:TripService
:ActivityService
:ExpenseService
:Database
selectTripFromList(tripId, userId)
viewTripDashboard(tripId, userId)
getTripSummary(tripId)
fetchTripBasicInfo(tripId)
returnTripDetails()
returnTripDetails()
par
getActivities(tripId)
fetchItinerary(tripId)
returnActivitiesList()
returnActivitiesWithTimes()
getExpensesAndBudget(tripId)
fetchExpenses(tripId)
returnExpensesList()
calculateTotalVsBudget()
returnBudgetStatus()
getMembers(tripId)
fetchMemberList(tripId)
returnMemberList()
returnMemberList()
aggregateDashboardData()
returnCompleteDashboardDTO()
renderDashboard()
displayItineraryMembersBudgetActivity()


:TripMember
:TripDashboard
:SOSController
:AlertService
:Database
:EmergencyContac
:SystemAdmin
clickSOSAlert()
showConfirmationDialog()
showConfirmationDialog()
confirmEmergency()
triggerSOS(userId, tripId, location)
processEmergency(userId, tripId, location)
fetchTripDetailsAndEmergencyContacts(tripId)
returnTripDataAndContacts()
prepareAlertMessage()
alt
sendEmailOrSMS(tripDetails, memberDetails)
returnDeliveryConfirmed()
[notifyAdmin]
sendSystemAlert(alertData)
returnAdminNotified()
logSOSEvent(userId, tripId, location, timestamp)
returnEventLogged()
returnAlertTriggeredSuccessfully()
returnAlertTriggeredSuccessfully()
showConfirmationAndInstructions()
showSuccessMessage("Emergency services alerted")
opt
[noEmergencyContactSet]
sendAlertToAdminOnly()
returnAcknowledged()


:User
:Dashboard
:AuthController
:SessionManager
clickLogout()
requestLogout(tokenOrSessionId)
invalidateSession(sessionId)
destroySession()
sessionDestroyed()
clearServerSideCache(userId)
logoutConfirmed()
clearLocalStorageAndCookies()
redirectToLoginPage()


TripMember
:Expense Controller
BudgetMonitorService
:TripService
Database
NotificationService
addExpense(amount, category)
validateTripBudgetExists(tripId)
getTripBudget(tripId)
getBudgetAmount()
IsBudgetExists()
saveExpense(expenseData)
isExpenseSaved()
triggerBudgetCheck(tripId)[cite: 1]
getAllExpenses(tripId)
getExpensiveList()
calculateTotalExpenses()
compareTotalToThreshold()
alt
[[total < threshold]]
isWithinBudget()
"Expense added normally"
[[total > threshold]]
sendBudgetExceededAlert(tripId)
isAlertSent()
isBudgetExceeded()
"Expense logged, WARNING: Budget exceeded!"


:user
:loginPage
:AuthController
:userService
:Database
openLoginPage()
showLoginForm()
enterEmailAndPassword(email, password)
clickLogin()
submitLogin(credentials)
authenticate(email, password)
fetchUserByEmail(email)
alt
[user not found]
noUserFound()
authenticationFailed()
showError("Invalid credentials")
displayErrorMessage()
[account locked]
userData + accountLockedFlag
accountLocked()
showError("Account locked")
displayLockMessage()
[user found and password correct]
userData + hashedPassword
verifyPassword(plainPassword, hashedPassword)
authenticationSuccess()
createSessionOrJWT(userData)[cite: 1]
createSessionOrJWT(userData)
redirectToDashboard()
viewDashboard()